Advanced Software Development

Johan Bontes (johan@digitsolutions.nl)

CSC3003S

Clean coding

Coding principles

- DRY: don’t repeat yourself
- OneTask: do only one thing in a function/class
- Meaningful names
- No littering
- No magic constants
- const correctness

DRY – don't repeat yourself

```
void doThis() {
  X -
  -
  -
  Y -
  -
  -
  Z -
  -
  -
}
```

```
void doThat() {
  X -
  -
  -
  Y -
  -
  -
}
```

```
void doThis2() {
  doX();
  doY();
  doZ();
}
```

```
void doThat2() {
  doX();
  doY();
}
```

OneTask: do one thing in a function

```
void doThis(a, b, c, d, e) {
  X -
  -
  -
  Y -
  -
  -
  Z -
  -
  -
}

void doThis2(a, b, c) {
  doX(a, b);
  doY(b);
  doZ(c);
}
```

Meaningful names

Code is written for humans first, computers second

Functions: verbNoun

Variables: two or more nouns

- include unit of measure (liters/secs/kg)
- Long scope => long name
  - For (i ...) only in loop scope => short name
  - for longer scopes use more descriptive name
- Limit the lifetime (scope) of variables
- Do not use Hungarian notation szFilename, ui32_counter
  - Use strong types instead
- Use consistent names (begin/end, isDone, ...)

No littering

Code is always up to date

Comments are not

- Try and put more descriptions into code
  - Fewer into comments
- Remove commented out code
  - Use the versioning system (git) instead
- Use comments to explain why,
  let the code explain how/what

No magic constants

Magic constants are naked numbers/strings in your code

```
a += 100; //why 100, not 101? what is 100?
if ('zorg' == name) { ... //hidden backdoor?
```

Name constants instead

- `const int LevelOffset = 100; //Levels are 100 units apart`
- `const auto superuser = 'zorg'; //definitely a backdoor`

- Makes the intention clear
- Constants defined in a single place (DRY)
  - Easy to update
- You can document named constants

Const correctness

Java: @ReadOnly, final

C++/Rust: const

Python: not supported

Allows for pure functions

- No side effects

const/final variables

- Easy to reason about
  - will keep initial value
  - simplifies debugging
  - compiler optimization will fold extra variables

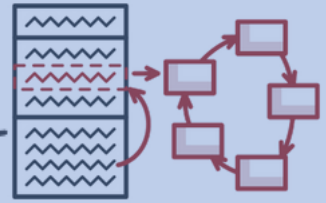
Behavioral patterns



Command
Action objects



Strategy
Object per algorithm



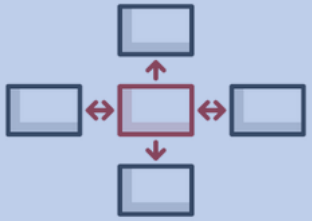
State
Finite state machine



Template
Algorithm blueprint



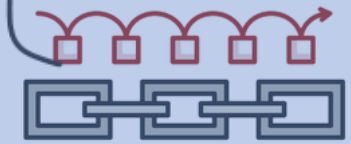
Memento
Snapshot/undo



Mediator
Reduce cross-talk



Observer
Subscribe to events



Chain of responsibility
Message passing



Visitor
Command++



Iterator
Visit every element

Iterator



Walk through every item in a collection

Collection: first + next + current methods

Iterator: `for (a in Collection) { a.dostuff() }`

=> for in loop runs the following pseudocode

```
c.First();                  //current = c[-1]
while (c.next()) {  //current = c[0,1,2,...]
   a = c.current(); a.dostuff();
}
```

Design custom collections with `first, next, current` methods

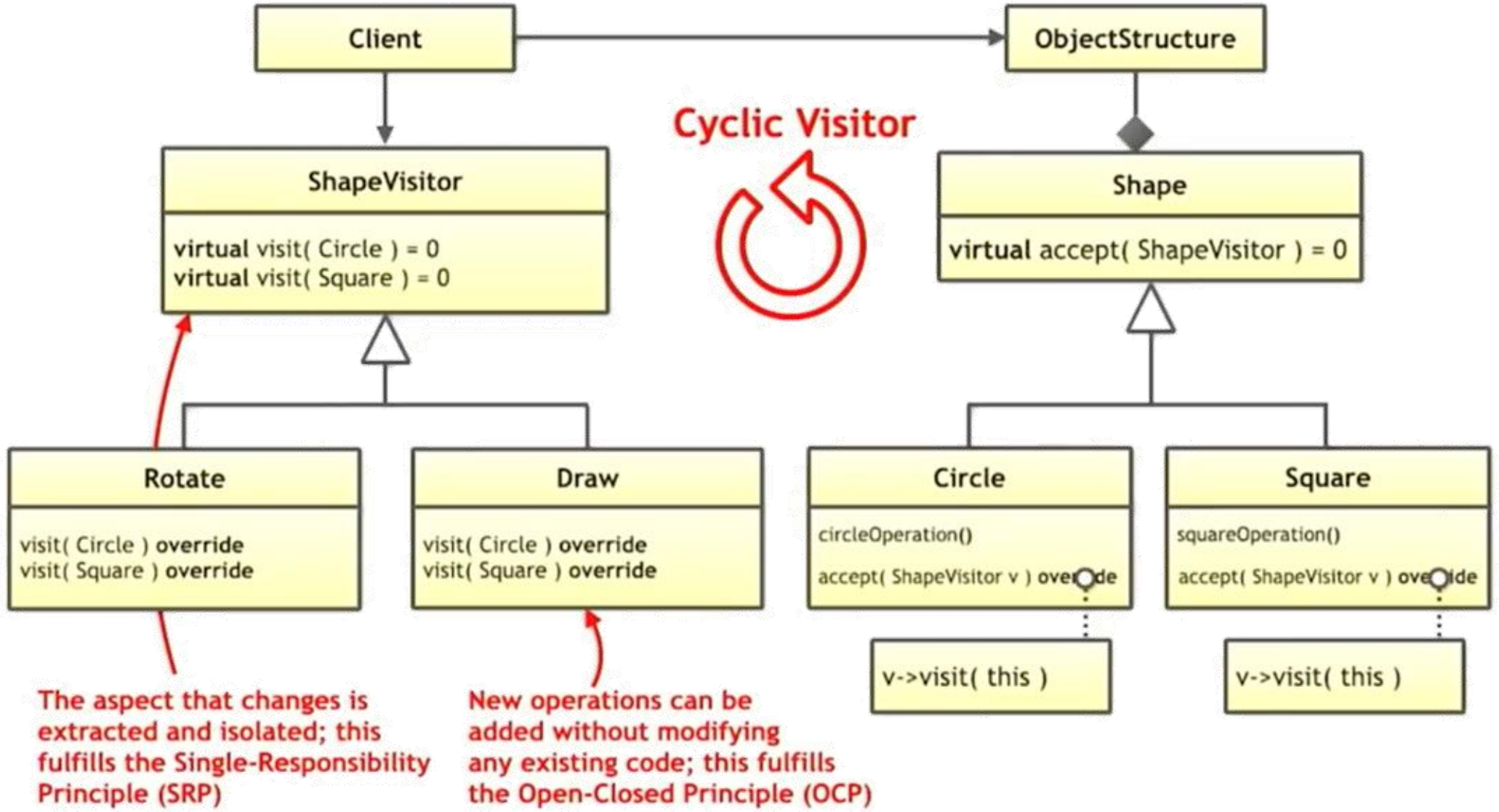
Every modern language has for in loops.
No longer needs an OOP workaround

Examples

- Builder: https://godbolt.org/z/rvMxz7q4Y
- Adapter: https://godbolt.org/z/G483PWoxa
- Decorator: https://godbolt.org/z/Ehv6Wdv1P
- Iterator: https://godbolt.org/z/e7M44ncb7
- Visitor: https://godbolt.org/z/a3raPo67P
- State: https://godbolt.org/z/3av9K56a3
- Chain of responsibility: https://godbolt.org/z/5456eG3x7
- ~~Strategy: https://godbolt.org/z/jT8Wf8cee~~
- ~~Observer: https://godbolt.org/z/q8croqeeP~~

GoF visitor. https://godbolt.org/z/T4qfEhY7T

Adds operations to a type hierarchy



Visitor pattern

In OOP it’s easy to add types

- Just add a class

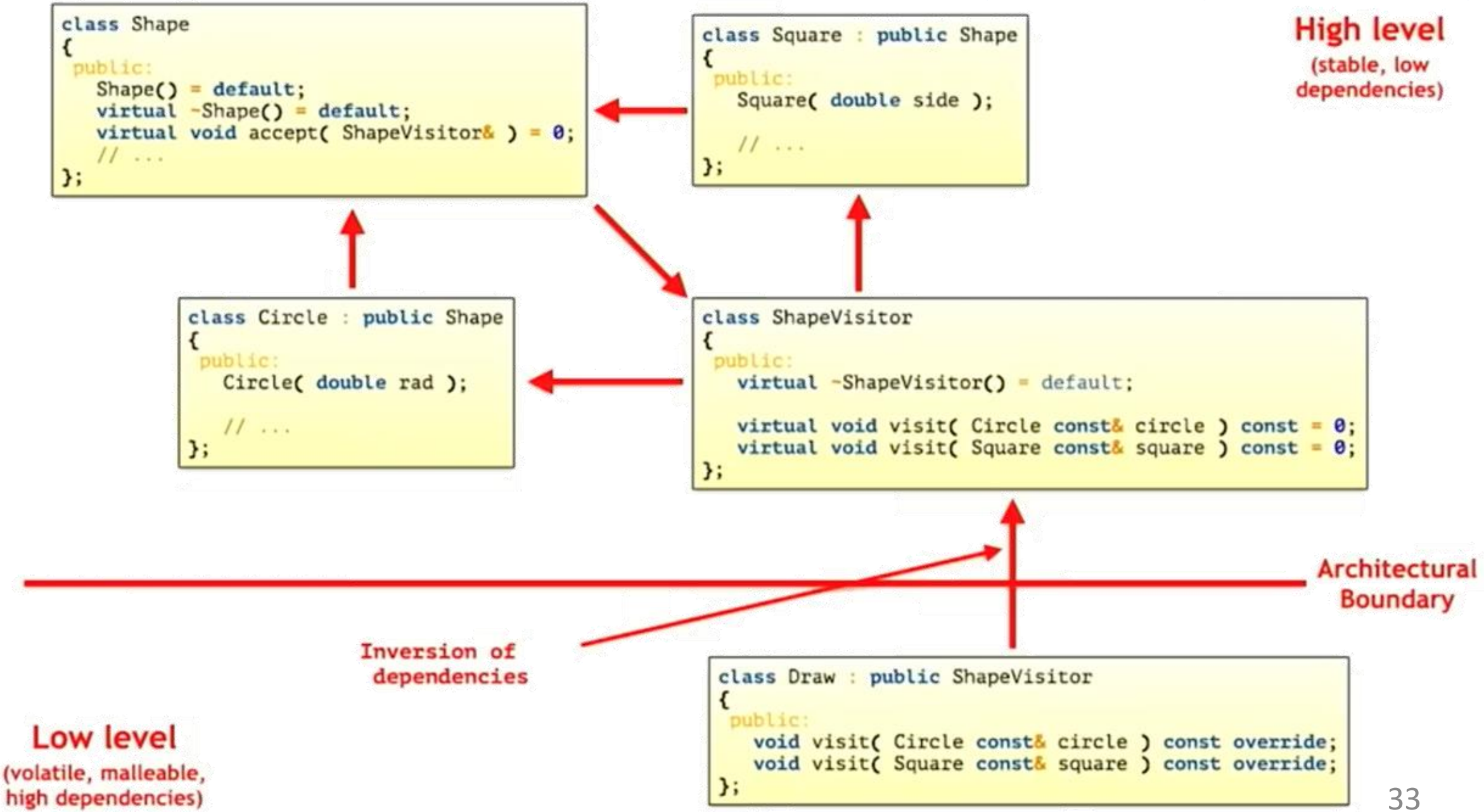
But it’s hard to add operations

- Operations are another dimension
- But a hierarchy is one-dimensional

Visitor pros - cons

Pros	Cons
Operations can be added easily (OCP)	Makes it hard to add new types (shapes)
Hides implementation details of operations (SRP)	Operations are limited to public interface of types
	Base class required (God object)
	Heap allocation needed
	Memory management needed
	Requires virtual functions (extra indirection)
	Has a cyclic dependency !!

The classic Visitor pattern has cyclic dependency



How about a procedural solution

Code demo: https://godbolt.org/z/qKz89fz8E

- Needs heap allocated objects
- Relies on switch statement
- Impossible to maintain

A modern solution: std::variant + std::visit

Code demo: https://godbolt.org/z/a4xGvchco

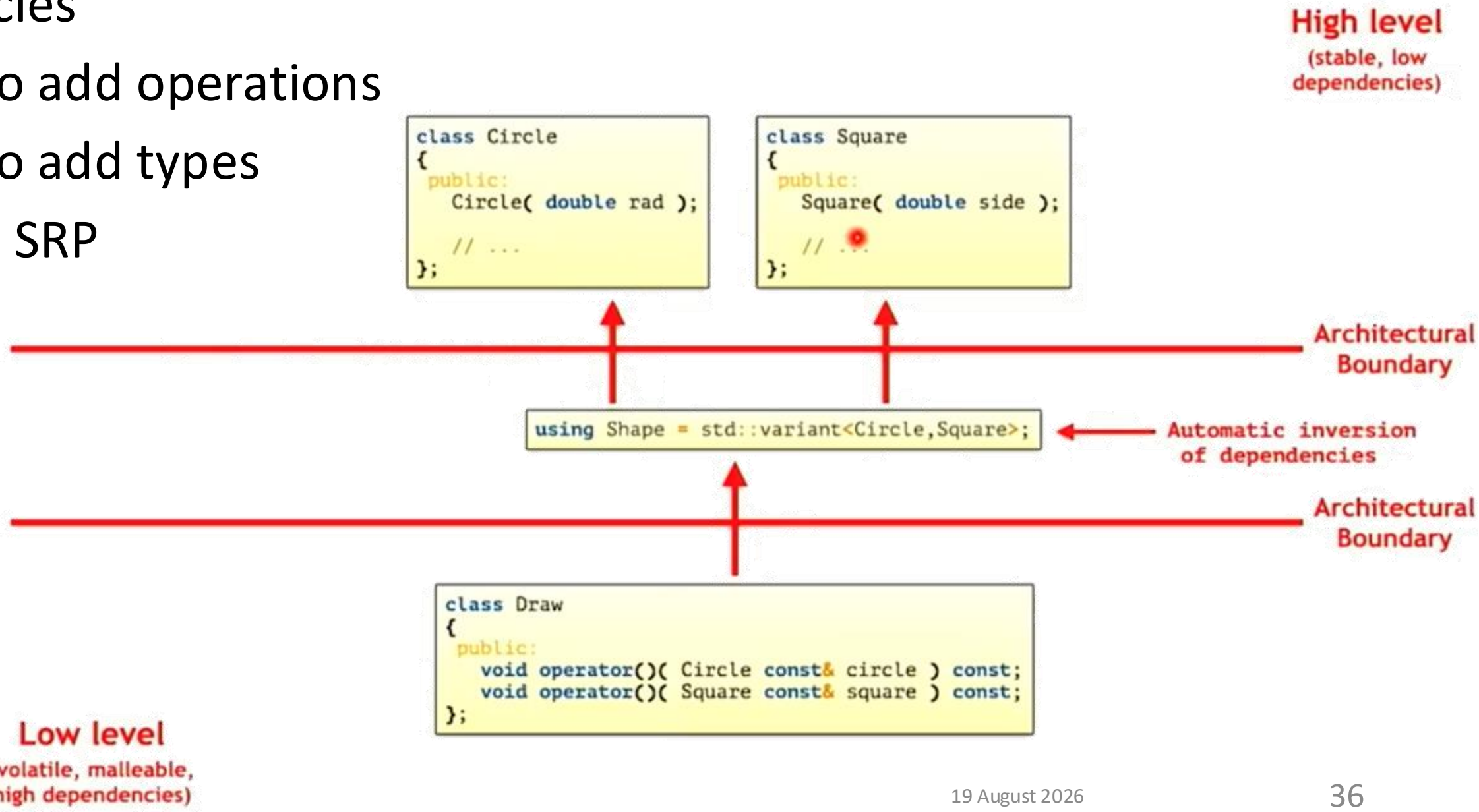
std::variant + std::visit

No cycles

Easy to add operations

Easy to add types

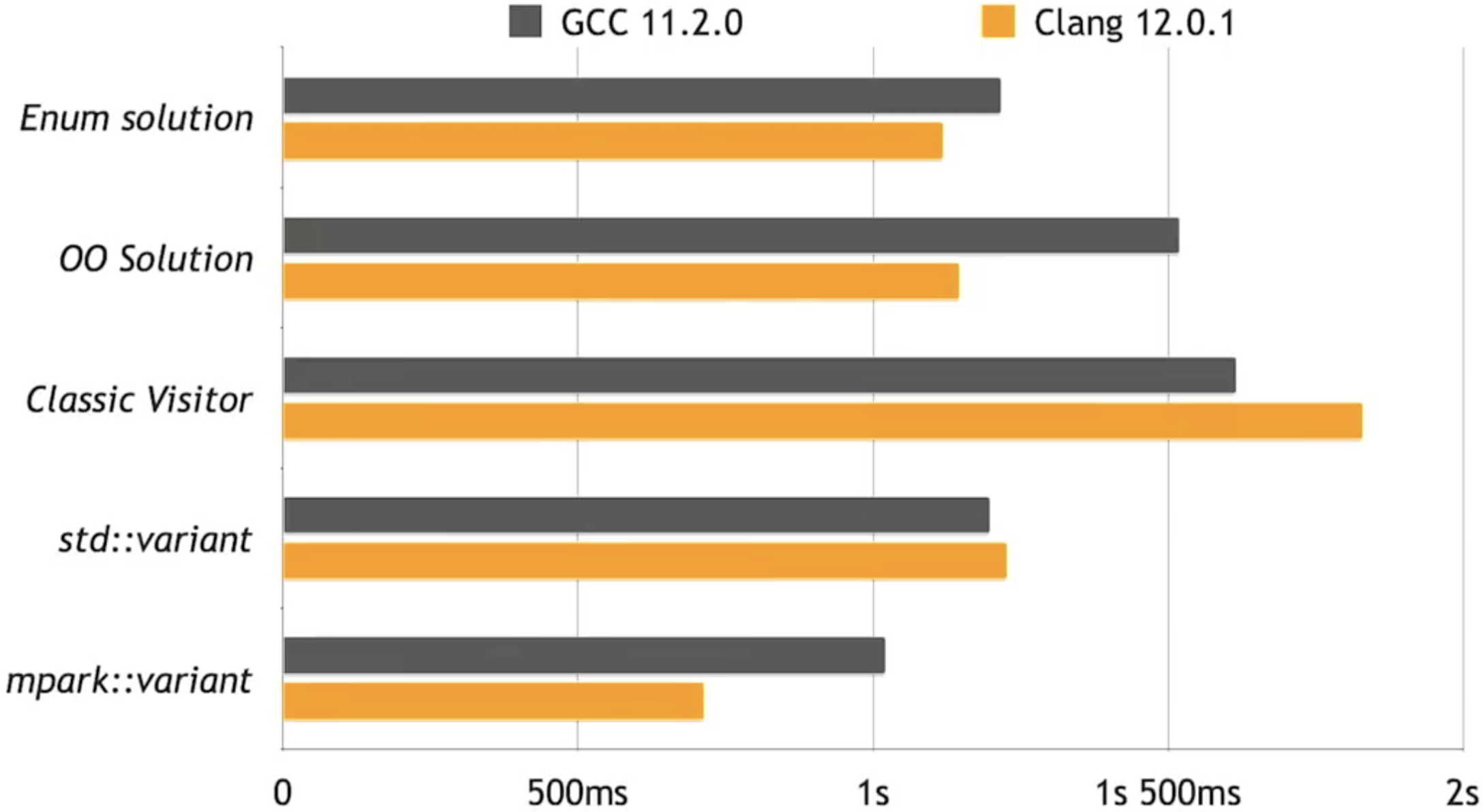
OCP + SRP



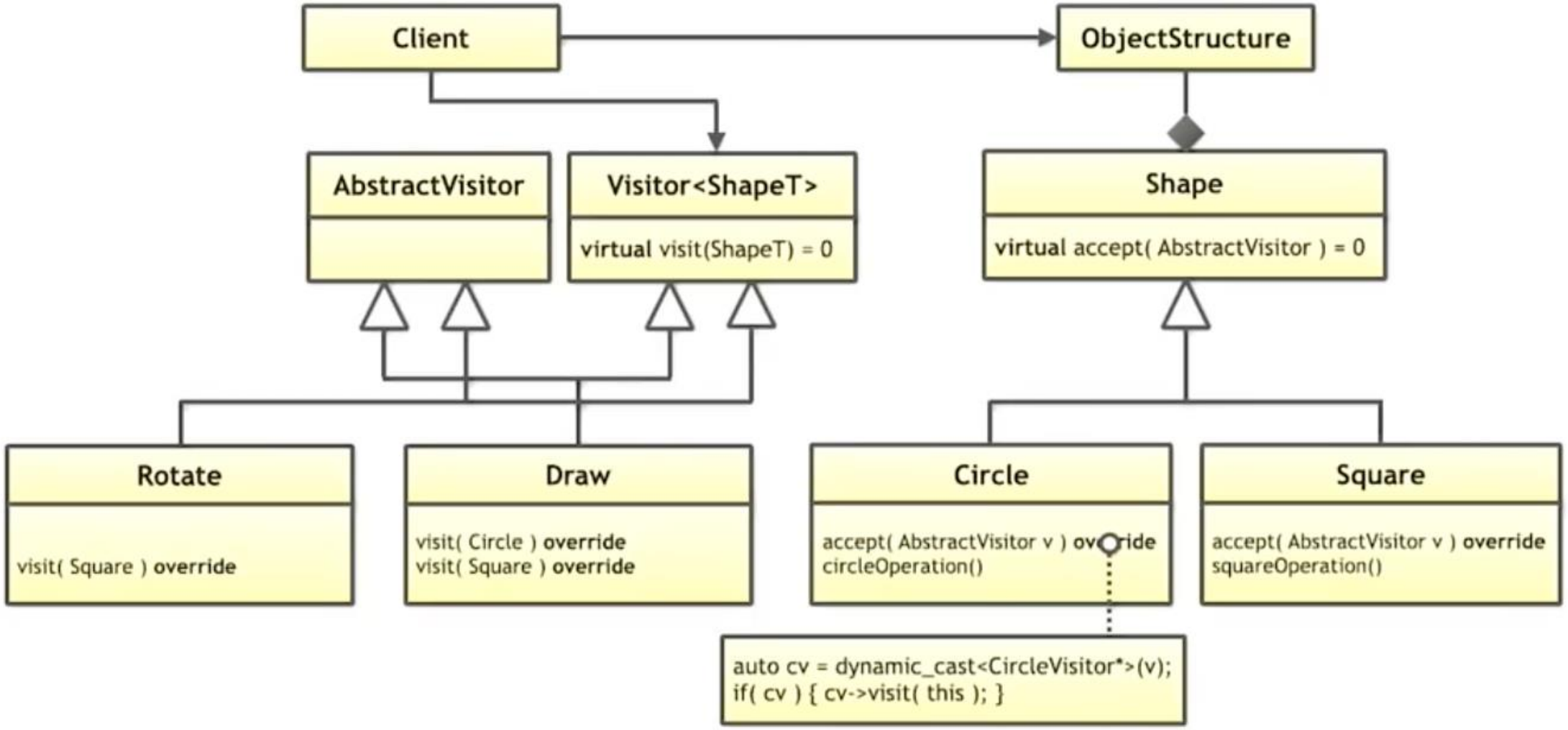
std::visit pros - cons

Pros	Cons
Operations can be added easily (OCP)	Mixing big + small types carries large overhead - Use proxy pattern to fix
Hides implementation details of operations (SRP)	std::variant is bad at hiding impl. details (info. hiding)
Easy to add types	
Operations can use all type details	
No manual heap alloc needed (auto destruction)	
No virtual functions	
No cyclic dependency	
Faster than OOP visitor	
Uses static polymorphism (faster, easier to optimize)	

Performance comparison

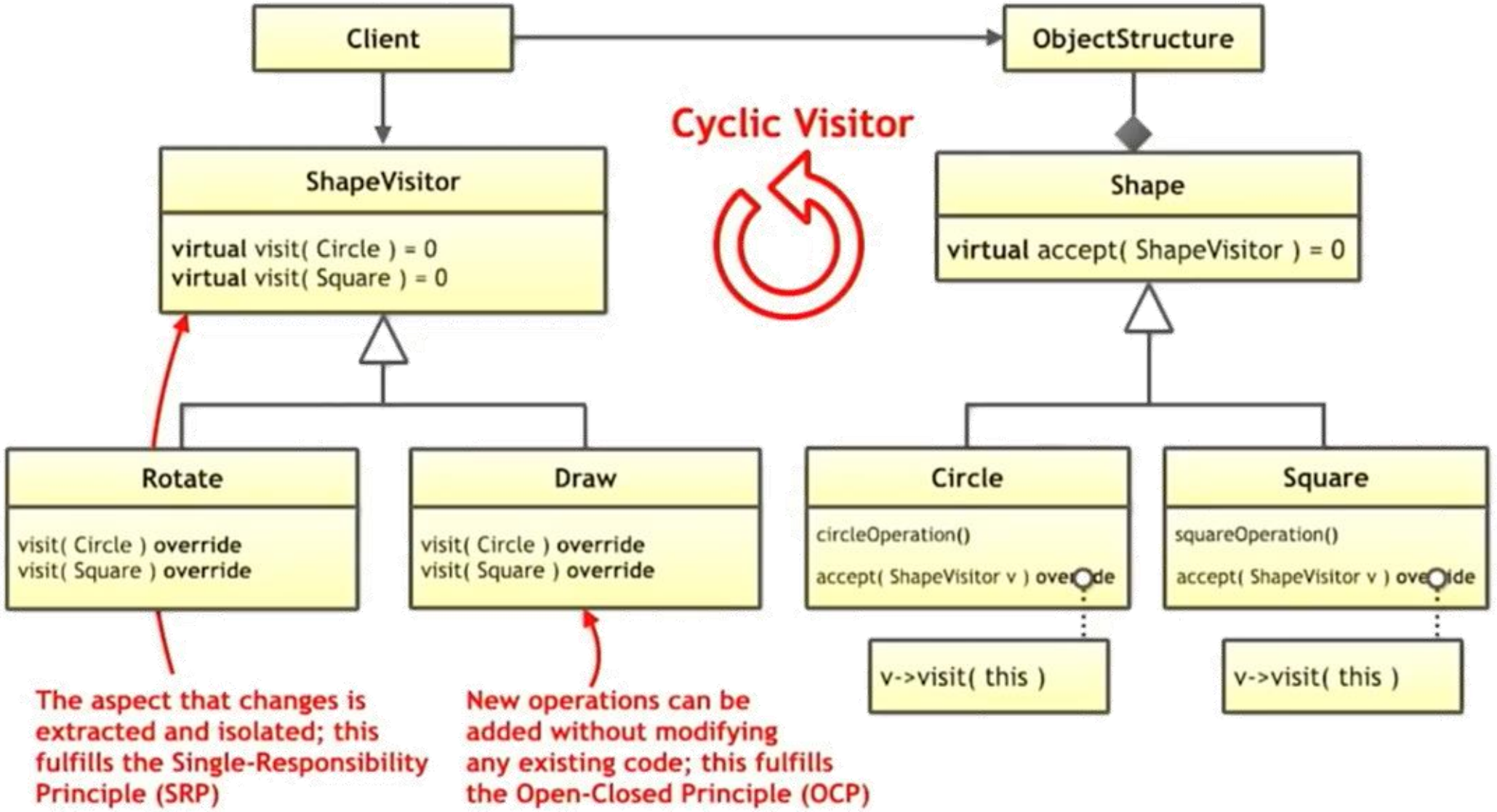


Acyclic OOP visitor pattern

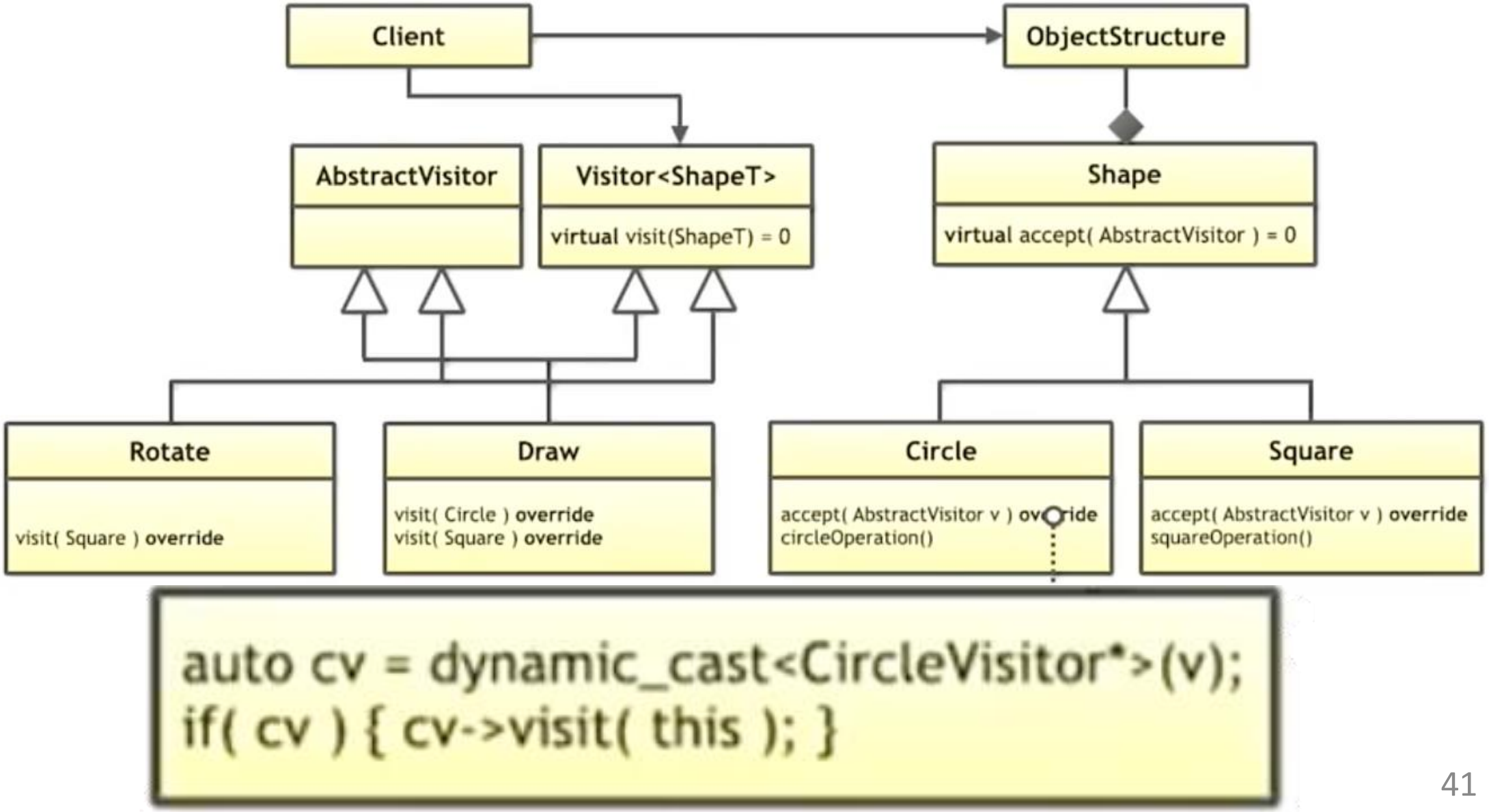


GoF visitor

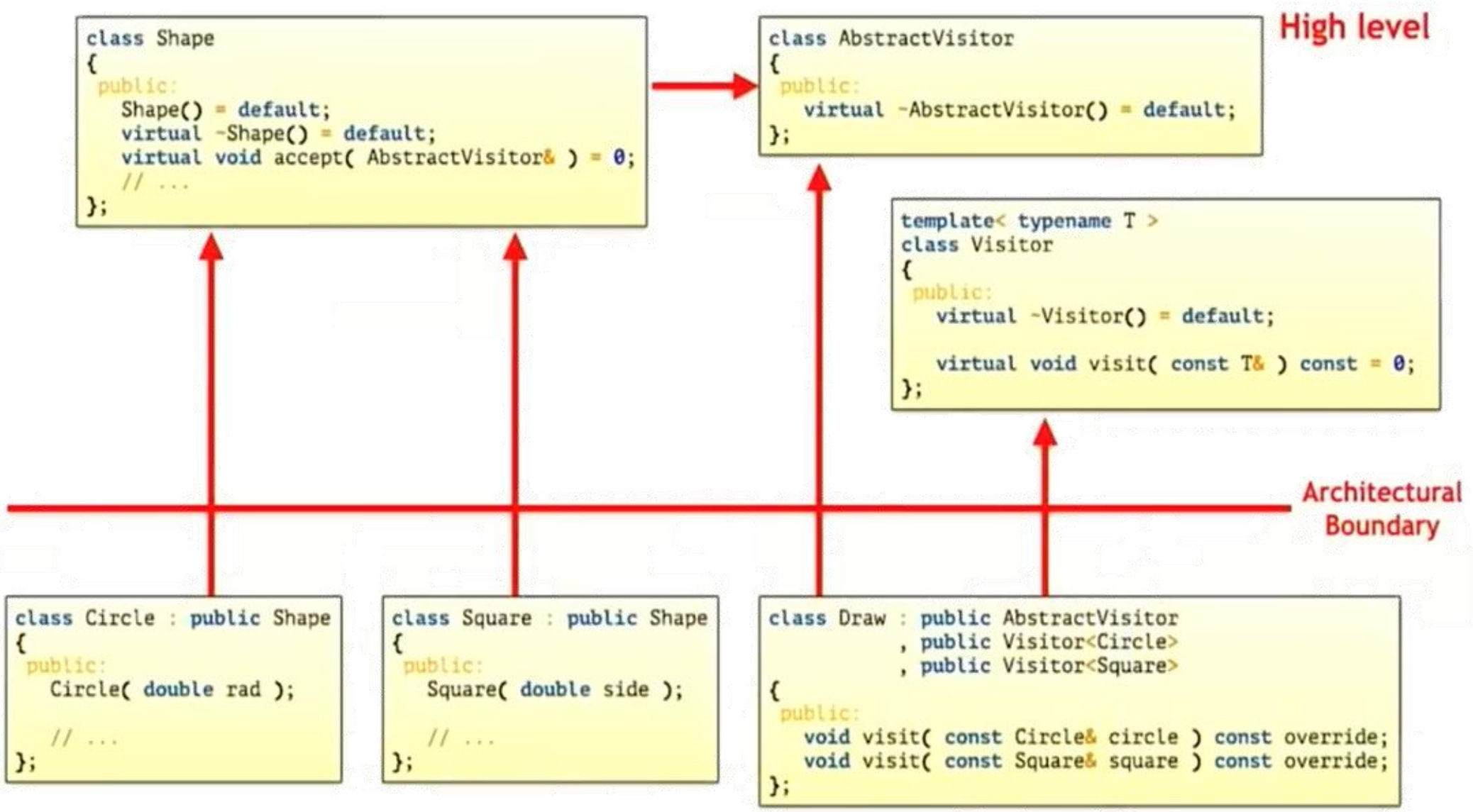
Adds operations to a type hierarchy



Acyclic OOP visitor pattern

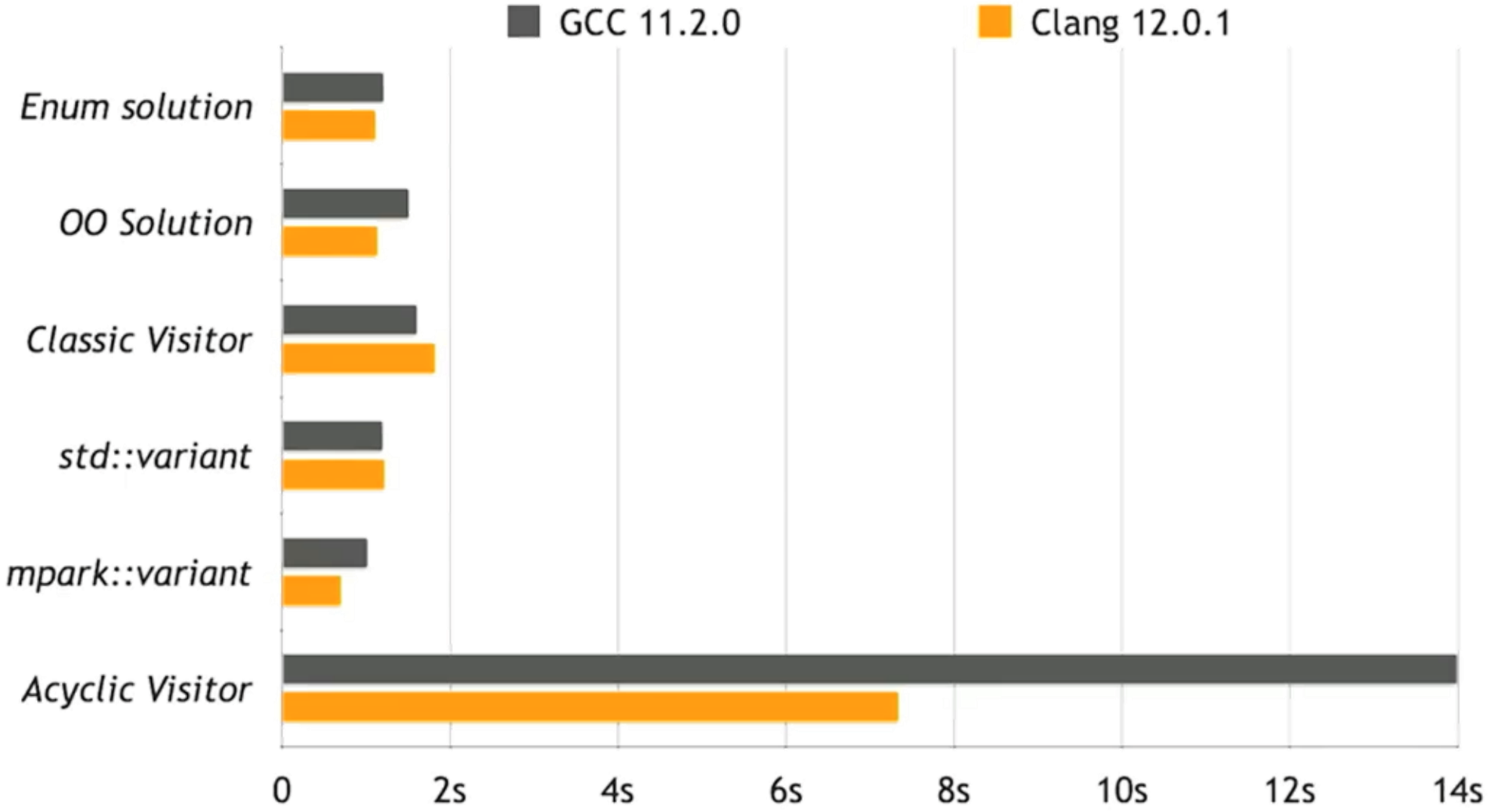


Acyclic visitor pattern



Acyclic OOP visitor

Drawback: slow, so very slow.



Recap

Visitor pattern

- Good if you want to add operations
- Bad if you want to add types

std::variant + std::visit

- Preferred in most scenarios

Acyclic visitor

- Also good for adding types
- Slow: use if overhead is not an issue
- Java: you have no choice

Memento

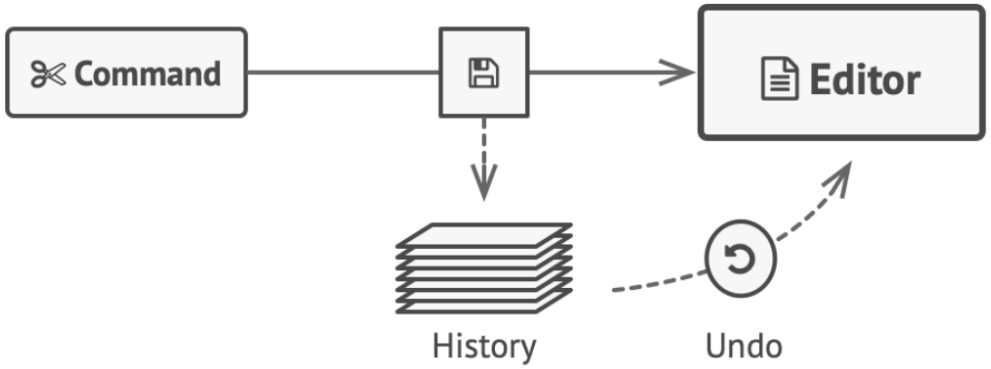


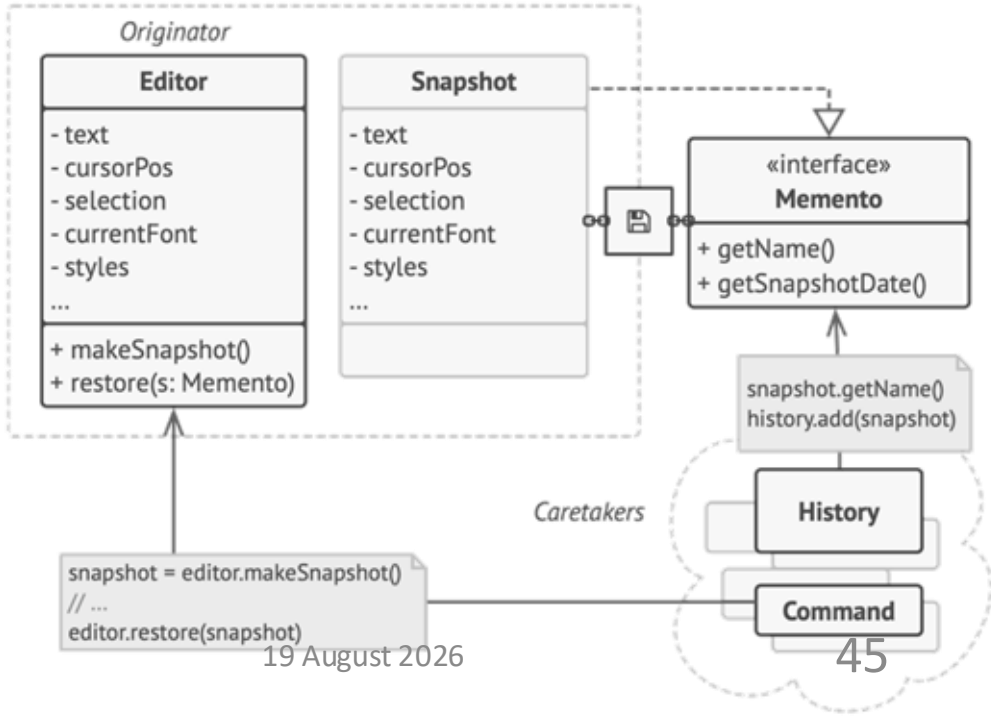
Program needs to save state from ≥ 1 objects

- Must not access object's private details
- Cloning objects not an option
  - Unsafe due to links with other objects

Solution

- Object creates '`snapshot`' to store state
- `Memento` adds metadata (date, command)
- `Caretaker` stores snapshots





Memento



```
class Editor implements IUndo {
  String text;   //gets saved
  Window owner;  //not saved
  EditorSnapshot makeSnapshot(); //IUndo
  void restoreSnapshot(EditorSnapshot restorePoint); //IUndo
}

class EditorSnapshot implements IMemento immutable {
  String text;
}

interface IMemento immutable {
  property int id;
  property IUndo source;
}

interface IMementoList immutable {  //if multiple objects must be restored
  property int id;
  property Datetime timestamp;
  property CommandId reason;
  property List<IMemento> sources;
  void restore() { for (s in sources) { s.source.restoreSnapshot(s); }
}

class Caretaker { //manages snapshots
  List<IMementoList> snapshots;
  void add(IMementoList item);
  void restore(int id) {
    restorepoint = getById(snapshots, id);
    restorepoint.restore();
```

Memento



Useful for

- Undo (revert back to previous savepoint)
- Transactions (commit/rollback)
- Save to file

Pro	Con
Can save state, without having to know internal details of objects	Can consume a lot of RAM if too many snapshots
Caretaker tracks snapshots	May need to track original object, if these are not permanent, to destroy obsolete snapshots
	Needs immutable objects